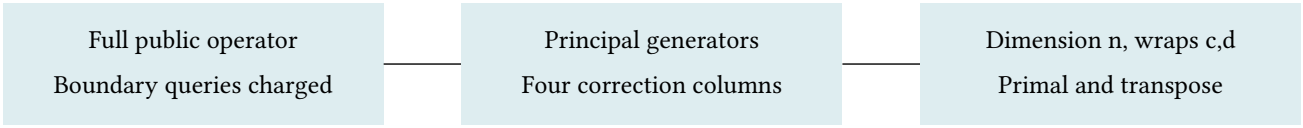
General companion block interfaces

Exact component panel · 2026-11-01 · companion-blocks-validation

Implemented and checked: arbitrary matrix dimensions over the fixed prime coefficient field, distinct companion wraps including zero wraps, primal and transpose application, four-column principal-block corrections, and conditional inverse-generator transport. All 738 context rows and 32,328 product checks passed. The independent dense references checked 3,072 principal identities and 192 inverse-generator identities. All 384 singular-oracle failures, 2,376 equal-wrap inputs and 67,920 deliberately corrupted results/counts or generators were rejected. Candidate and tuple-input A/A matched exactly.



Dense matrices and explicit inverse oracles occur only in the checker.

Editable diagram data and renderer are retained here. Production API: companion-blocks/operators.py. Derivation: companion-blocks/DERIVATION.md.

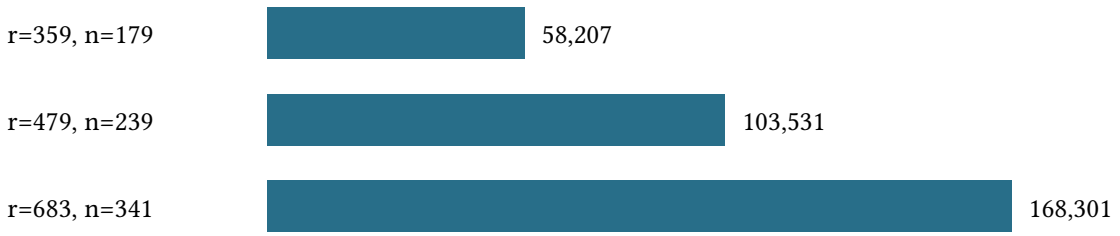
General operator. Let L=Z_(n,a), R=Z_(n,b), with a different from b in F_r. L^n=aI and R^n=bI imply (a-b)A=sum_j L^(n-1-j)GK^{t}R^j for j=0 through n-1. No restriction n=r or n less than r is needed. Two ordinary packed polynomial products per generator column implement either direction. The synthetic panel includes n=r+1 and n=1 as well as dimensions below r.

Principal generators. The leading block uses the restricted old generators and four explicit boundary outer products. A public operator cache acquires the last full row, crossing column, last block row and first full column. Each miss executes and charges a full source application. The checker compares every cached vector to its independently constructed dense row or column. For four cuts and four new wrap choices, all 64 requests, 10 misses and 54 hits per source context are checked; 10r field slots are retained.

Inverse transport scope. For an invertible A, R A^{-1} - A^{-1} L = (-A^{-1}G)(A^{-t}K)^t. The wrapper transforms generators through supplied forward and transpose inverse oracles. This experiment supplies explicit unit-upper-triangular reference inverses and verifies both inverse identities before use. It establishes the swapped-wrap transport, not efficient inverse synthesis. Singular reference-oracle exceptions must propagate.

Frozen cohort. Each arm retains all 81 earlier generator contexts, their four cuts and four new wrap pairs. The 288 synthetic contexts cross primes 7,13,19,43; dimensions 1,2,3,r-1,r,r+1; wraps (2,1),(1,2),(0,1),(1,0); and upper-triangular, rank-deficient diagonal and zero matrices. Synthetic proper half-size blocks also test the correction formula. Full operators use zero, ones, seeded and all small basis vectors (endpoints for larger sizes). Restrictions use every basis vector at sizes at most 19, otherwise one seeded vector, in addition to entrywise displacement identities.

Exact work and execution custody



Candidate source instances18,27,36; new wraps2,1; exact counts.

Three recorded source instances illustrated from the full retained panel; all raw subchecks remain in runs/cases.jsonl. Exact counts have no uncertainty bars. Query, acquisition, native and storage units remain separate.

- r=359, n=179: 58,207 field products, 87,206 additions, one inversion, 326 native products, 466,832 packing bytes.
- r=479, n=239: 103,531 field products, 155,156 additions, one inversion, 434 native products, 829,808 packing bytes.
- r=683, n=341: 168,301 field products, 252,188 additions, one inversion, 494 native products, 1,347,632 packing bytes.

Boundary acquisition totals, once per source context. These include all 10 full operator queries shared across the 16 principal restrictions:

- r=359: 1,142,030 field products, 3,180 native products, 9,132,960 packing bytes, 3,590 retained field slots.
- r=479: 2,041,070 field products, 4,260 native products, 16,324,320 packing bytes, 4,790 retained field slots.
- r=683: 3,321,350 field products, 4,860 native products, 26,555,040 packing bytes, 6,830 retained field slots.

Formulas checked. At dimension n and width s, each direction uses $2s(n-1)+n$ field products, $s(3n-2)+1$ additions and one inversion; $2s$ native products and $4s$ input encodings. With byte digit width $d=\mathrm{ceil}(\mathrm{bitlength}(n(r-1)^2)/8)$, packing is $4snd$ bytes, product serialization and decoding are each $2s(2n-1)d$ bytes, and reductions number $2s(2n-1)$. The principal correction alone costs n products and n+1 additions; boundary acquisition is separately charged. Inverse transport records s calls in each oracle direction and ns negations. No oracle is assigned zero total cost; its supplied dense reference implementation is outside a fast-solve claim.

Predata custody. Commit 91aaadd; nine source locks and one public input archive. Before any numerical trial, seed-preflight.py checked every one of the 738 protocol execution cells against the expanded plan. The earlier companion-matvec export failure remains unchanged. This panel uses a frozen row index binding offsets and row hashes; the analyzer verifies that index before exporting, and the full raw archive is hashed in the export file list.

Execution. One timeout 900 invocation of ../.audit-venv/bin/python -u companion-blocks-validation/check.py, no restart, producer exit 0. All 738 planned contexts completed, none excluded or dropped. Exact timestamps and shared-host metadata are in runs/. No valid wall-time, CPU or RSS comparison was collected. Export validation status, lock verification and PDF inspection are recorded in provenance.json and export-check.log.

Evidence. runs/summary.json; runs/cases.jsonl; runs/row-index.json; runs/export.json; expected-summary.json; protocol.json; source-lock.json; input-lock.json; seed-preflight.json; launch.json. Report source, chart data, SVGs and renderer are retained beside this PDF.

Derived next step: Schur generators of width s+2

This page records a derivation and unexecuted prototype. The prototype in companion-schur/schur.py passed syntax checking only; no Schur numerical experiment ran. Full proof and call schedule: companion-blocks/SCHUR_NEXT.md.

Partition A=[[E,B],[C,D]], with E invertible, leading dimension n and trailing dimension m. Set $S=D-CE^{-1}B$, $U=[-CE^{-1},I]$, $V=[-E^{-1}B;I]$. Then UA=[0,S] and AV=[0;S]. Multiplying the full displacement identity by U and V gives $(L22-CE^{-1}L12)S-S(R22-R21E^{-1}B)=(UG)(V^tK)^t$.

The companion crossing terms are $L12=a\ u_n\ v_m^t$ and $R21=u_m\ v_n^t$, where u is the first and v the last basis vector. For distinct new wraps c,d, grouping the wrap and crossing corrections yields generators

$GS = [\ UG \mid aCE^{-1}u_n+c\ u_m \mid -S\ u_m\ ]$,

$KS = [\ V^tK \mid S^tv_m \mid B^tE^{-t}v_n+d\ v_m\ ]$.

Thus $Z_{(m,c)}S-SZ_{(m,d)}=GS\ KS^t$ with constructed width s+2. This does not assert minimal displacement rank or require S itself to be invertible.

Counted implementation path. A Schur application uses two full A applications and one E solve: apply A to [0;x], solve Ez=Bx, apply A to [z;0], then return Dx-Cz. The transpose path uses two A^t applications and one E^{-t} solve. Columnwise generator construction, including both Schur endpoint vectors, has an explicit uncached schedule of 2s+6 source applications and s+2 leading-block solves in each direction, plus scalar updates and generator storage. These are nested calls, not unit-cost solves. Batching and the recursive pivot policy are still required.

Required obstruction control. A singular E does not establish that A is singular. The invertible matrix [[0,1],[1,0]] has zero leading scalar block. The next implementation must preserve that distinction and record a block pivot obstruction or execute a separately charged pivot policy.

Next frozen gate. Validate the Schur prototype using the checked block API with explicit reference E oracles. Check dense Schur identities, both directions, complete nested call accounting, corrupted corrections, zero/singular Schur cases and the invertible-A/singular-E counterexample. Then replace reference oracles with a recursive backend.

Literature context. Bostan et al. (2017), Section 6.2, also derives low-width Schur generators under its displacement conventions and generic rank-profile assumptions. The displayed formula specializes the project's conventions. No base-field randomized success guarantee is imported. Primary source, SIAM J. Matrix Anal. Appl. 38(3),733–775.